The Gateway and Providers: Custody, Routing, Failure and Cost

Learning series, document 3 of 6. 14 October 2026. Repository: `origin/main` at 980ffaf4e. Written for Choir's owner, to teach the general ideas of a platform gateway and to show exactly how Choir's gateway implements them, where it is weak, and what the choices are. No live host was touched for this document: every claim is read from the code, from an existing Choir document, or from a vendor's published behaviour, and is labelled that way. Mutation class of this document: **green** (docs only). Almost every change it discusses would be **red** (gateway and provider calls, identity, routing).

Evidence labels: **code** (read in the repository at the commit above, with a path), **doc** (an existing Choir report or problem record, linked), **upstream** (vendor behaviour as recorded in a Choir evidence file) and **inference** (reasoning that has not been tested; treat it as a hypothesis).

How to read this

Section 1 explains why a gateway exists at all and draws the request path. Sections 2 to 8 each take one job of the gateway and follow the same arc: a primer on the general idea, how Choir does it (with files and functions), where it is weak (with the incidents that showed it), and the options with their trade-offs.

- Section 2: who is calling (guest identity and credentials).
- Section 3: which model answers (catalog, policy, pools).
- Section 4: what happens when a provider fails (retries, circuit breakers, failover).
- Section 5: how long anyone waits (timeouts and deadlines).
- Section 6: Jev, the typed judge.
- Section 7: the search plane (fan-out, cooldowns, caching, budget routing).
- Section 8: what it all costs (metering and prompt caching).

Section 9 walks one request end to end and gives a debugging checklist. The glossary, self-check and reading list close the document. Sibling documents cover the store (01), the desks that make these calls (02), the VM lifecycle that issues credentials (04), publication (05) and a field guide to trade-offs (06).

The short answer

1. **The gateway is a chokepoint by design.** Guests never hold a provider key; they hold a per-computer bearer token and call one host service, `cmd/gateway` on port 8084, which holds every model and search credential, enforces per-computer rate limits, trips circuit breakers, and logs every call (code: `cmd/gateway/main.go`, `internal/gateway/handlers.go`). A chokepoint is where custody, policy and metering are cheapest to enforce, and where one bug hurts everyone.
2. **Guest identity is a bearer plus a source-address check, not a bound credential.** The token is `computerID:random`, stored by the gateway only as a SHA-256 hash, delivered to the guest on a root-only 8 MiB credential disk, and valid for 720 hours with no rotation. Only the Jev route pins a bearer to one guest address; everywhere else a leaked bearer works from any Choir guest (code: `internal/gateway/config.go`; doc: the isolation review §3.5).
3. **Routing has three layers of indirection:** a role (management, research...) maps to a spec in a computer-owned policy file, a pool spec hashes the thread onto one pool member, and the gateway maps provider/model to an adapter. Indirection is what let the owner reroute the live paper during an outage by editing one file (code: `internal/modelpolicy/model_policy.go`, `internal/modelcatalog/catalog.go`).

4. **Choir learned what a circuit breaker should count the hard way.** Three incidents in five days: client errors (400s) opened a shared breaker; one account's usage limit stopped four roles; a free model's 429s opened the breaker for a paid model on the same provider. The current design keys breakers by (provider, model), counts only provider-side failures, and walks the pool forward on failover (code: `internal/gateway/circuit_breaker.go`, `internal/toolregistry/toolloop.go`).
5. **Errors cross process boundaries as strings.** Retry, failover and search cooldown decisions are made by matching text such as `"circuit open"`, `"429"` and `"quota"`. It works, and it is the most fragile contract in this part of the system.
6. **Search fans out, caches, and now routes by budget.** About 4,000 searches a day; free providers first, one paid provider at a time on shortfall, per-provider daily caps counted in SQLite, identical searches answered from a 30-minute cache or joined in flight (code: `internal/gateway/searchplane/`; doc: the search efficiency report).
7. **Jev turns judgement into typed questions.** A pinned, cheap model answers `choice`/`score` questions with a probability distribution; the runtime uses it for citation support, intake labels and front-page importance, off the desks' critical path (code: `internal/gateway/jev.go`, `internal/agentcore/support_judge.go`).
8. **Cost is metered in the guest, priced at list rates, and mostly uncached.** Tokens flow from provider usage fields into run events and `internal/llmcost`; `/api/costs` reports per computer. The gateway logs tokens but keeps no ledger. A per-second timestamp in every desk's system prompt still defeats cross-turn prompt caching at this commit (code: `internal/agentcore/tool_profiles.go:207`; doc: the prompt cache analysis).

1. Primer: why a gateway sits between guests and providers

1.1 The three jobs: custody, policy, metering

A Choir computer is a Firecracker VM that runs model-written code. Its desks need two kinds of outside service: **inference** (model calls) and **search** (web search APIs). Each provider wants a credential: an API key, an OAuth token, a subscription login. There are three places those credentials could live:

option	what it means	why it fails or works
in each guest	every VM holds the API keys	any code in any guest (including model-written cells) can read and exfiltrate keys that are worth money to everyone
in the browser	the user's client calls providers directly	the browser is not where agents run; keys leak to any page script
in one host service	guests call a **gateway**; the gateway holds keys	one place to protect, rotate, meter and limit; one place that can fail

Choir chose the third. The general pattern has several names: an **API gateway**, a **credential broker**, an **egress proxy**. Whatever the name, it does three jobs:

1. **Custody.** The secret never leaves the gateway. Callers get a *capability to use* the secret (a scoped bearer), not the secret. If a guest is compromised, the attacker can spend through the gateway until the bearer is revoked, but cannot take the key elsewhere.
2. **Policy.** Every call passes one point where you can say no: rate limits per caller, refusal of unknown models, circuit breakers on sick providers, daily spend caps.
3. **Metering.** Every call passes one point where you can count tokens, searches and money, keyed by the caller's identity.

The price of a chokepoint is **fate-sharing**: if the gateway is down, slow, or misjudges a provider as unhealthy, every computer suffers at once. Several incidents in this document are exactly that.

1.2 The request path in Choir

```
guest computer (Firecracker VM, 10.200.x.2 or 172.x.0.2)
+-------------------------------------------------------------+
| desk run (tool loop, internal/toolregistry/toolloop.go)     |
|   -> gatewayruntime.Provider (internal/gatewayruntime)      |
|   -> hostlink transport: resend refused dials up to 3 min   |
|   Authorization: Bearer <computerID>:<random>               |
|   token read from RUNTIME_GATEWAY_TOKEN_FILE                |
|   (credential disk, mode 0400)                              |
+-------------------------------------------------------------+
      | tap network, host side x.x.x.1:8084
      v
host: go-choir-gateway (cmd/gateway, all interfaces, port 8084)
+-------------------------------------------------------------+
| authenticateAutoputer: hash(token) == registry hash?        |
|                        peer IP is a Choir guest address?    |
| PerAutoputerRateLimiter: 60 calls / 1 min window / computer |
| resolveProvider: provider name + catalog check              |
| CircuitBreakingProvider: provider breaker + model breaker   |
| provider adapter (Responses / Chat Completions / Anthropic) |
+-------------------------------------------------------------+
      | HTTPS, credentials from gateway-provider.env, ~/.codex
      v
chatgpt, opencode-go, opencode-zen, bedrock, zai, fireworks;
OpenRouter (Jev); 9 search providers incl. local SearXNG
```

Caddy, the only public edge, refuses `/provider/*` and `/internal/*` with a 403 (code: `nix/node-b.nix:251-256`), so the gateway's routes are not reachable from the internet. The gateway listens on all interfaces (`SERVER_HOST=0.0.0.0`, `nix/node-b.nix` gateway unit) because guests reach it through their tap's host-side address.

The routes are registered in one place (code: `RegisterRoutes`, `internal/gateway/handlers.go:1074-1091`): inference (`/provider/v1/inference`), an OpenAI-compatible adapter for third-party clients such as Zot (`/provider/openai/v1/chat/completions`, `internal/gateway/openai_compat.go`), Jev judgements (`/provider/v1/judgments`), search and its health, credential issue and ensure, and breaker inspection and reset.

2. Who is calling: guest identity and credentials

2.1 Primer: bearers, binding, rotation and ambient authority

A **bearer token** is a credential that works for whoever bears it. HTTP's `Authorization: Bearer <token>` is the common form. Four properties decide how safe a bearer is:

- **Storage at rest.** The server should store a *hash* of the token, not the token, so a leak of the server's database does not leak usable credentials. A random 32-byte token has enough entropy that a plain SHA-256 is adequate; passwords need slow hashes, random tokens do not.
- **Delivery.** How the token reaches its holder matters as much as the token. Kernel command lines, environment variables and world-readable files are all readable by more processes than you intend.
- **Lifetime and rotation.** A short-lived token limits the damage of a leak; rotation (issuing a new token and revoking the old) bounds it further. Long-lived tokens with no rotation are convenient and turn every leak into a long one.
- **Binding.** A pure bearer works from anywhere. A **bound** credential also requires proof that the caller is the intended holder: a client certificate (mutual TLS), a signature over each request, or at least a check that the request comes from an expected network address. Binding is what makes a stolen token useless elsewhere.

Its opposite is **ambient authority**: being trusted because of *where* you are rather than *what you hold*. "Any process on 127.0.0.1 is the operator" is ambient authority. It is easy and it means every process in that network namespace holds the credential.

2.2 How Choir does it

Issuance. When vmctl cold-boots a computer it calls the gateway's loopback-only issue route (code: `issueGatewayTokenAt`, `internal/vmctl/ownership.go:1597`; called before `BootVM` at `ownership.go:1376` and `:2099`). The gateway's `IdentityRegistry` (code: `internal/gateway/config.go:194`) generates 32 random bytes, stores only `TokenHash` (SHA-256, hex) with `IssuedAt`, `ExpiresAt` and `Active`, revokes any earlier identity for the same computer, and returns `computerID + ":" + token` once (`IssueCredential`, `config.go:281`). The registry is persisted to `gateway-identities.json` by write-to-temp then rename, mode 0600 (`persistLocked`, `config.go:237`; path from `GATEWAY_IDENTITY_STORE_PATH`, `nix/node-b.nix:1016`). Write-then-rename is the standard way to make a file update atomic: a crash leaves either the old file or the new one, never half of each.

Delivery. vmmanager writes the token into a small ext4 image, `credential.img`, built from a staging directory whose files must be regular, root-owned and mode 0400, then formatted with `mkfs.ext4 -d` and renamed into place mode 0600 (code: `createCredentialDisk`, `internal/vmmanager/manager.go:2587-2660`). Inside the guest the runtime reads the path in `RUNTIME_GATEWAY_TOKEN_FILE`, falling back to the `RUNTIME_GATEWAY_TOKEN` variable (code: `GatewayToken`, `internal/provideriface/config.go:291`). This replaced the original channel, the kernel command line, which any guest process could read from `/proc/cmdline` (doc: s1-gateway-token-on-kernel-cmdline).

Validation. Every inference, search and Jev call runs `authenticateAutoputer` (code: `handlers.go:644`): parse the bearer, look up the computer, refuse if revoked or expired, compare the hash, then require that the TCP peer is loopback or a Choir guest address (`isAuthorizedRuntimePeer`, `handlers.go:717`). A Choir guest address is `172.x.0.2` or `10.200-215.x.2` (`isChoirGuestPeer`, `handlers.go:740`), matching the tap allocation.

Reconciliation after a gateway restart. Before persistence existed, a gateway restart forgot every hash. The ensure route lets vmctl hand back a computer's existing raw token so the gateway re-imports it (code: `HandleEnsureCredential`, `handlers.go:591`; `EnsureCredentialForPeer`, `config.go:336`). It refuses a token whose hash conflicts with a known one, or one that is revoked or expired.

The Jev pin. The Jev route goes further: the first time a computer calls it, `BindJevPeer` (`config.go:437`) records the caller's guest IP, and later calls from any other address are refused. Because a computer gets a new guest address on each boot, only the host can move the pin, through the ensure route with a `peer_ip` (code: `config.go:336-399`; doc: l2a-merge-recall-on-the-paper). That is the right shape for binding: **the party that knows the truth (vmctl, which assigned the address) moves the pin; the party being checked (the guest) cannot.**

Operator routes. Credential issue and ensure, search health and reset, and breaker inspection and reset all require `isAuthorizedCredentialCaller`: a loopback peer *and* the header `X-Internal-Caller: true` (code: `handlers.go:710`, header constant at `handlers.go:126`).

Provider credentials. Model and search keys come from `/var/lib/go-choir/gateway-provider.env` (code: gateway unit `EnvironmentFile`, `nix/node-b.nix`). ChatGPT is different: it uses a ChatGPT login whose OAuth access token is refreshed from a refresh token in a Codex-style `auth.json`, 45 minutes before expiry (code: `ChatGPTAuth`, `needsRefresh`, `internal/provider/provider.go:2726-2835`). Providers are registered only if their credentials exist (code: `ResolveAll`, `provider.go:3098`). Error bodies from providers are drained and, for 4xx, a redacted and truncated reason is logged but never returned to the caller (code: `logRejectedRequest`, `internal/provider/rejected_request.go:19`), and the gateway applies a second redaction pass before answering (`sanitizeError`, `handlers.go:672`). Two layers of redaction is **defence in depth** applied to secrets in error messages.

2.3 Where it is weak

- **Bearers are not bound, except on Jev.** Any Choir guest address passes `isChoirGuestPeer`, so a bearer stolen from computer A works from computer B (doc: isolation review §3.5, "tokens are bound to transport, not to the VM"). The harm is spend and rate budget under another computer's name, and anything the provider sees as that computer.

- **720-hour lifetime, no rotation.** The default TTL is one hour (`DefaultAutoputerTokenTTL`, `config.go:64`), but staging sets 720 h because nothing rotates tokens (`nix/node-b.nix:1019-1022`, with the comment saying so). A snapshot resume or recovery passes an empty token and reuses the old one (code: `ownership.go:1365`). *Inference:* a computer that stays up, or resumes from a snapshot, more than 30 days after its last cold boot would find its bearer expired, and this reading found no path that re-issues it; worth a test before it becomes an outage.
- **Loopback is a credential.** The operator routes trust any loopback process that sends a header anyone can send. Every host process, including the VMMs, SearXNG (which parses untrusted pages) and the Go module proxy, can mint or re-import identities and reset breakers (doc: isolation review §3.5, severity High for loopback as a class).
- **Test networks are accepted in the production path.** `isAuthorizedRuntimePeer` also admits the three documentation ranges `192.0.2.0/24`, `198.51.100.0/24` and `203.0.113.0/24` (`isDocumentationTestPeer`, `handlers.go:752`), so that unit tests can use `httptest` defaults. The isolation review flags the same pattern in vmctl. *Inference:* exploiting it needs a process able to source traffic from those addresses on the host, so the risk is low, but test conveniences in an authentication check are a pattern to remove.
- **A second custodian.** The front-importance timer loads the same `gateway-provider.env` and calls OpenRouter directly with the gateway's transport code (code: `nix/node-b.nix:681`; `internal/frontimportance/scorer.go:103-110`). It is a host process, so custody stays on the host, but its calls bypass the gateway's rate limits and per-computer metering.
- **Hash comparison is not constant-time.** `ValidateCredential` compares hex strings with `!=` (`config.go:402-435`). Because both sides are hashes of the token, a timing leak reveals bits of a hash, not of the token. *Inference:* not exploitable in practice; still, `subtle.ConstantTimeCompare` costs nothing and is the habit to keep.

2.4 Options and trade-offs

option	what it buys	what it costs
generalize the Jev pin to every route	a stolen bearer fails from any other VM; vmctl already moves the pin on boot	one more registry write per boot; must handle address changes during snapshot restore
short TTL plus rotation at every boot and resume	bounds a leak to hours	vmctl must write a new credential disk or push a token; a missed rotation is an outage
vsock instead of TCP for guest-to-host	the peer CID is set by the VMM, so the guest cannot spoof identity; no firewall dependence	new transport in every client; Firecracker vsock plumbing
mutual TLS with a per-computer client certificate	standard, network-native binding	certificate issuance and rotation; TLS inside the host
unix socket with `SO_PEERCRED` for operator routes	the kernel tells the server the caller's uid; no header to forge	needs host services to run as distinct users first (isolation review §4.4)
route front-importance through the gateway	one custodian, one meter	the timer depends on the gateway being up

The isolation review recommends the first and the fourth row as the near and far steps (doc: isolation review §4.6).

3. Which model answers: catalog, policy and pools

3.1 Primer: indirection, consistent assignment, last known good

Hard-coding "research uses model X" in code means every routing change is a deploy. Platforms add **indirection** in layers, each owned by whoever should change it:

- a **catalog** (platform-owned): which models exist, on which provider, with which limits;
- a **policy** (owner-owned): which logical role uses which model or group of models;
- a **pool**: an ordered group of interchangeable models, so load spreads and one model's failure has a fallback.

When a pool assigns members to work, it should do so **consistently**: the same conversation should land on the same model, because providers cache prompts per model (section 8) and because changing models mid-thought changes behaviour. The usual tool is to hash a stable key (the thread or conversation id) onto the members. That is a simple form of **consistent assignment**; full consistent hashing additionally minimizes reshuffling when members are added or removed.

A configuration file that users can edit will sometimes be invalid. A robust loader falls back to the **last known good** configuration rather than failing the work, and says so.

3.2 How Choir does it

The catalog is code: `SupportedModels` lists each model's id, provider, output limit and context window, and `Pools` defines the pools (code: `internal/modelcatalog/catalog.go:42`, `:211`). At this commit the default pool is, in order:

1. `opencode-go/deepseek-v4.1-flash`
2. `opencode-go/cat-jam-free`
3. `chatgpt/gpt-6-luna`

LongCat left the pool on 14 October after measuring 21.8 s median and 115 s p90 per call against 2.5 s and 10 s for gpt-6-luna; the catalog comment cites `docs/evidence/model-latency-paper-2026-10-14.md`. Note a small drift: the state-and-vision report (§3.6) says every desk runs on deepseek and luna; the code still includes cat-jam-free as the second member (owner, 13 October: free models join the pool to measure them).

The policy is a computer-owned TOML file, `System/model-policy.toml`, that maps roles to specs. The platform default sets every role to `pool:default`, with reasoning effort high for Texture and management and low for research and the conductor (code: `defaultPolicyText`, `internal/modelpolicy/model_policy.go:457`). The file carries a version line; a file without the current version line is rewritten at load, and a file containing `# owner-edited` is left alone (`ensureDefaultPolicyFile`, `:523`). `Load` (`:96`) parses the file and, if it is invalid, returns the last valid policy for that path, or the platform fallback, together with an error recorded in run metadata. That is last-known-good in practice.

Assignment happens when a run starts. `EnrichMetadata` resolves the role to a selection, computes a thread key, and calls `Assign` (`:136-180`). `ThreadKey` (`:185`) is the Texture agent id (`texture:<doc>`) for Texture runs, so a document keeps one model; otherwise the trajectory id; otherwise a random key. `Assign` (`:208`) hashes the key with 32-bit FNV-1a, rotates the pool to start at that member, and returns the first member plus the rest as ordered fallbacks. A concrete model the catalog has retired is remapped to the default pool (`assignment = "retired_remapped"`). The management *task* role is pinned to one non-preview model and gets no fallbacks, because a failover could land on the free model the pin exists to avoid (comment at `:154-156`). All of this is written into run metadata (`llm_spec`, `llm_assignment`, `llm_thread`, `llm_fallbacks`), so a trace shows how a model was chosen.

At the gateway, `resolveFromMultiProvider` (code: `internal/gateway/handlers.go:500`) takes the provider name the guest sent and refuses a model that the catalog does not list for a catalogued provider. Without a provider name it falls back to model-name heuristics: a `glm-` prefix goes to Z.AI, anything containing `claude` or `anthropic` to Bedrock. Each provider adapter speaks one of three wire shapes: OpenAI Responses (chatgpt), OpenAI Chat Completions (opencode-go for most models, fireworks) or Anthropic Messages (bedrock, zai, some opencode-go models) (code: `openCodeWireShapeForModel`, `provider.go:1220`).

3.3 Where it is weak

- **Two of three pool members share one provider.** When opencode-go's breaker was provider-wide, a cat-jam-free rate limit took deepseek down with it (doc: free-model-429-opens-provider-circuit-stub-stories). Keying breakers per model fixed the coupling in the gateway, but a provider-level outage (account, network, billing) still removes two members at once. A pool is only as diverse as its least-shared dependency.

- **Hashing per trajectory scatters caches.** Management, research and engineering hash their trajectory id, so the same persistent desk lands on different models over time, and each model's prompt cache goes cold (doc: prompt cache analysis, breaker B5).
- **Owner-edited files freeze.** The `# owner-edited` escape hatch is what let the owner reroute the paper during the 14 October incident without a deploy. It also means later platform defaults never reach that file; the problem record notes that the paper's policy must be restored by hand once the fix is live.
- **Heuristic routing by name** (`glm-` prefix, `claude` substring) is a second source of truth beside the catalog. *Inference:* harmless while the catalog is small; it becomes a surprise when a model name collides.

3.4 Options and trade-offs

option	gain	cost
sticky model per persistent desk (hash the agent id, as Texture does)	warm caches; consistent behaviour per desk	coarser load spreading; one desk's model failure hits all of its work
pools that span providers by construction	correlated provider failure cannot empty a pool	fewer cheap models to choose from
policy overlays with expiry ("pin luna until the fix deploys")	incident edits clean themselves up	a little more policy machinery
delete name heuristics; require provider in every request	one source of truth	callers that omit provider fail loudly (that is the point)

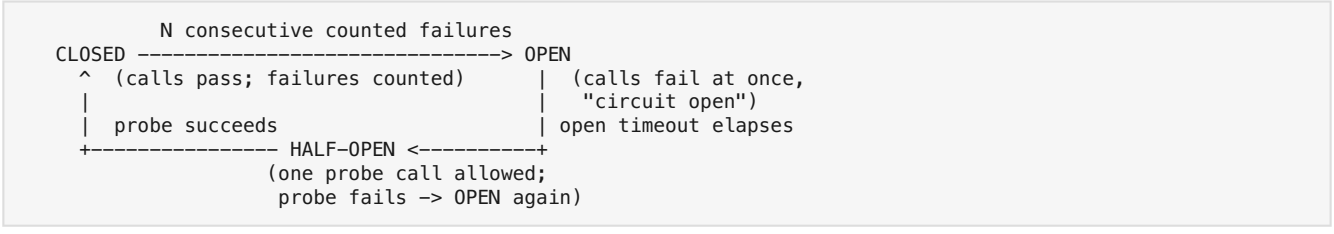
4. When providers fail: retries, circuit breakers and failover

4.1 Primer

Retries turn transient failures into success, but they multiply load. If a client retries 3 times and the layer above it retries 4 times, one logical call can become 12 upstream calls: **retry amplification**. During an overload, retries make the overload worse. The standard defences:

- retry only what is **retryable** (a refused connection, a 503, a 429 with a known wait), never a request the server rejected as malformed;
- **back off** exponentially with **jitter** (random spread), so clients do not retry in lockstep;
- honour **`Retry-After`** when the server sends it;
- retry at **one** layer, or budget retries across layers.

A **circuit breaker** stops calling a dependency that is failing, so the caller fails fast and the dependency gets room to recover. It is a small state machine:



Two design questions decide whether a breaker helps or hurts:

1. **What counts as a failure?** Only evidence that the *dependency* is unhealthy: 5xx, timeouts, refused connections, rate limits. A 400 (we sent a bad request) or a caller cancelling says nothing about the dependency. Counting them lets one bad caller open the breaker for everyone.
2. **What is the breaker's key?** Too coarse (one breaker per provider) and one sick model blocks healthy ones; too fine and each caller must discover an outage separately. This is the **bulkhead** idea from ship design: compartments, so one flood does not sink the ship.

Failover moves work to an alternative after the primary fails. It must not loop (A to B to A), must try each alternative at most once, and must not fire on the caller's own cancellation.

4.2 How Choir does it

The breaker is `health.CircuitBreaker` (code: `internal/health/circuit_breaker.go`), configured in `cmd/gateway/main.go` with `FailureThreshold: 5`, `OpenTimeout: 30s`, `HalfOpenMaxProbes: 1`. `ExecuteClassified` (`circuit_breaker.go:263`) takes a function that decides whether an error counts.

`CircuitBreakingProvider` (code: `internal/gateway/circuit_breaker.go`) wraps each provider with **two levels** of breaker:

- one breaker **per model**, which counts what `providerFailureCounts` (`:169`) says counts: everything except a caller cancellation and a 4xx other than 408 (timeout) and 429 (rate limit). A deadline overrun still counts: the model was slow.
- one breaker **per provider**, which counts only `providerUnreachable` (`:191`): a transport failure (`": http call: "` in the error) that is not a timeout and not the caller leaving. No model escapes a provider you cannot reach.

`execute` (`:130`) nests the model breaker inside the provider breaker. An open model breaker returns `"provider P model M: circuit open (model unhealthy)"`, deliberately without a status code, because the guest reads a `429` in the text as "wait and retry" and `circuit open` as "fail over now" (comment at `:142–144`). `BreakerRegistry` exposes every breaker as `provider` and `provider/model` for the loopback ops routes.

The guest's retries (code: `gatewayruntime.Provider.call`, `internal/gatewayruntime/provider.go:179`):

- up to 3 attempts, 0.5 s and 1 s apart, for transport errors, 5xx and truncated bodies;
- a gateway 429 with `Retry-After` is waited out, up to 10 waits of at most a minute each, without spending an attempt (`maxRateLimitWaits`, `maxRateLimitWait`, `:175–176`); this came from the research fan-out incident below;
- an error containing `circuit open` returns at once: retrying an open circuit only delays failover.

Above that, the tool loop retries a rate-limit error three more times after 5, 20 and 60 seconds (`providerRateLimitRetryDelays`, `internal/toolregistry/toolloop.go:226`).

Failover (code: `toolloop.go:492–525`): when an error is still failing after its retries and `IsModelFailoverError` (`:1053`) says it is a rate limit, usage limit, quota, 5xx, timeout, unknown model or open circuit, the run moves to the next fallback from the pool, emits a `model_failover` retry event, suffixes the conversation id with `#failover`, and strips provider-specific reasoning replay from the history (one provider's reasoning items are not valid input to another). It walks fallbacks forward and never back, so each member is tried at most once; never on `context.Canceled`.

Per-computer rate limit: `PerAutoputerRateLimiter`, a fixed window of 60 requests per minute per computer (code: `internal/gateway/ratelimit.go`), with separate buckets for search and for each Jev scope (`rateLimitBucketKey`, `handlers.go:1012`) so search bursts cannot starve inference.

4.3 The incidents that shaped this

date	what happened	lesson
10-10	One computer's nine `400 Bad Request` responses opened the shared opencode-go breaker; another computer's Texture run failed with "circuit open" (inference-breaker-trips-on-client-errors)	count only provider-side failures; log the 4xx reason, which was being discarded
10-11	Eleven ChatGPT 429s opened the chatgpt breaker; conductor, management, research and both verifiers all routed there, so four of six roles stopped (chatgpt-usage-limit-stops-research-and-management)	one account is one failure domain; roles need fallbacks on another provider

date	what happened	lesson
10-11	Research fan-out made 239 child activations; the 60-per-minute limiter rejected 2,546 calls; children retried blind and died; Texture made one call in five minutes (research-fanout-overruns-computer-model-budget)	a rate limit is a schedule the client must respect; honour `Retry-After`
10-14	cat-jam-free 429s opened the provider-wide opencode-go breaker; deepseek on the same provider failed; failover stopped at the first fallback; 29 research runs failed and stub stories published (free-model-429-opens-provider-circuit-stub-stories)	key breakers by (provider, model); fail over through the whole pool; hold uncited revisions

The last row is the clearest example of a **cascading failure**: a free model's upstream rate limit became a provider-wide breaker, which became failed research, which became a Texture revision saying "no verification work was performed", which auto-published to readers. Every link was a reasonable local decision.

The 10-14 record also does the retry arithmetic. Each runtime call could be up to three gateway calls (the guest retried a 502), and the tool loop retried a `429` text three more times, so one rate-limited thread kept hitting the same model for minutes (doc: same record, "Confirmed from code"). The fixes removed the 502 retry on `circuit open` and made the open model breaker the short backoff.

4.4 Where it is still weak

- **Errors are strings.** `providerFailureCounts` parses `": status (\d{3})"`, `providerUnreachable` looks for `": http call: "`, `IsModelFailoverError` matches a list of phrases, and the guest checks for `"circuit open"`. A provider that words an error differently, or a refactor that changes a format string, silently changes retry and failover behaviour. *Inference:* this is the most likely source of the next breaker incident.
- **`Retry-After` from upstreams is dropped** by the provider layer's sanitized errors, so the 30-second open model breaker stands in for it (comment, `circuit_breaker.go:32–36`; residual in the 10-14 record).
- **No jitter.** The guest's backoff is `attempt × 500 ms` and the tool loop's is fixed at 5, 20, 60 s (code). *Inference:* many runs failing at once retry in step; jitter is a one-line change.
- **Breaker state is per gateway process and in memory.** A restart closes every breaker. That is acceptable (it errs toward trying) but means a deploy during an outage briefly sends full load at a sick provider.
- **The fallback chain inherits the pool's provider concentration** (section 3.3).

4.5 Options and trade-offs

option	gain	cost
typed error envelope on the gateway wire (`class: rate_limited / quota / unreachable / rejected / circuit_open`, `retry_after_ms`)	retry and failover decisions read a field, not a sentence	both sides change; old guests need the text fallback for a while
plumb upstream `Retry-After` into the model breaker's open timeout	waits exactly as long as the provider asks	parse per provider; clamp to sane bounds
full jitter on every backoff	spreads retry storms	none worth naming
adaptive concurrency per (provider, model) (AIMD: add one on success, halve on 429)	the gateway learns each account's real limit and queues instead of failing	a queue in the gateway is more state to drain on deploy
one retry layer (gateway or guest, not both)	removes amplification by construction	the remaining layer must be smarter

5. How long anyone waits: timeouts and deadlines

5.1 Primer

Every call across a network needs a bound, or a hung dependency hangs its caller forever. When calls nest (guest to gateway to provider), the bounds must be ordered: **each inner layer must give up before the layer above it**, or the inner layer finishes work nobody is waiting for. The clean way is **deadline propagation**: the outermost caller sets one deadline, and each layer passes the remaining time down (Go's `context.Context` carries it). Fixed, independent timeouts at each layer are the common, fragile alternative.

Two other tools matter here. **Asynchrony** removes the wait entirely: return a handle now, deliver results as they arrive. **Hedging** sends a second request after a delay and takes the first answer, trading cost for tail latency.

Long timeouts have a cost too: anything that must drain them (a deploy, a restart) waits that long.

5.2 How Choir does it

layer	bound	where
gateway inference handler	10 min	`inferenceTimeout`, `internal/gateway/handlers.go:20`
gateway HTTP write timeout and shutdown drain	10 min 30 s	`SERVER_WRITE_TIMEOUT`, `SERVER_SHUTDOWN_TIMEOUT`, gateway unit in `nix/node-b.nix`
gateway unit stop timeout	11 min	`TimeoutStopSec`, `nix/node-b.nix:975`
guest gateway client	10 min 30 s	`gatewayClientTimeout`, `internal/gatewayruntime/provider.go:35`
guest resend of refused dials	up to 3 min, within the context	`RestartWait`, `internal/hostlink/hostlink.go`
each search provider call	30 s	`RequestTimeout`, `internal/gateway/searchplane/config.go:182`
one search stream	request timeout + pace wait + escalation window + 1 s	`search_plane.go:149`
guest search client	no per-attempt bound; the caller's context ends it	`NewGatewayClientFromEnv`, `internal/search/search.go:153`
Jev transport	2 min	`jevTransportTimeout`, `internal/gateway/jev.go:19-24`
support judgement call	90 s; queue wait 2 min	`internal/agentcore/support_judge.go:57-61`

Inference is ordered correctly: the provider call is bounded by 10 min, the guest waits 10 min 30 s, so the gateway's sanitized error reaches the guest instead of an EOF (comment in the gateway unit).

`hostlink` is worth studying as a small, correct idea: a **refused dial never reached the host**, so resending it is safe for any method, even a non-idempotent one. Anything that reached the host, including a reset mid-call, is returned to the caller as it was (code: package comment, `internal/hostlink/hostlink.go`). That distinction, "did the request arrive?", is the whole of safe retrying.

Search is now **asynchronous**. `choir.SearchStart` returns as soon as the gateway names the providers it started; each provider's results are forwarded to the desk worker as their own frame as they land, and wait in the session's search store for the next cell (code: package comment and `startSearch`, `internal/agentcore/desk_search.go:17-30`, `:105`).

5.3 Where it was weak, and what changed

On 11 October a research cell reported "the search timed out at the search gateway" while the gateway logged the same search as a success after 30.4 seconds. The guest's client timeout, each provider's timeout and the gateway handler's bound were all 30 seconds, so anything near the limit succeeded after the guest had given up (doc: guest-

search-timeout-equals-gateway-provider-timeout). A measurement found 51 of 509 searches (10%) ran exactly 30.0 s at the gateway. The first proposal was a **deadline ladder** (provider about 10 s, gateway about 25 s, guest about 35 s). The owner chose async search instead (12 October), which removes the tie rather than ordering it. This is a good example of the "prefer deletion" instinct from CLAUDE.md: the fix removed a wait instead of tuning three numbers.

The long inference bound has its own cost. A deploy stops every changed unit and starts none until all stops finish; the gateway's graceful drain of up to 11 minutes kept proxy, auth and corpusd down for that long, and the site answered 502 for up to 11 min 13 s (doc: deploy-takes-staging-down-during-gateway-drain). The site part is repaired; model calls are still refused during the drain (open item D3 in that record). A drain that protects one service by holding all the others hostage is a fate-sharing bug, not a timeout bug.

5.4 Options and trade-offs

option	gain	cost
propagate the run's remaining deadline in a request header	inner layers stop exactly when the caller will	every hop must read and honour it
async inference (submit, then poll or stream events)	deploys and restarts no longer cut calls; no long-held connections	a job store in the gateway, which is the planned-restart problem in a new place
shorter provider timeout with hedging to a second model	better tail latency	pays twice for hedged calls
drain the gateway separately from other units	site stays up during a drain	deploy orchestration work

6. Jev: typed judgements

6.1 Primer: System One questions

Many decisions an agent system makes are not writing; they are **classification**: is this sentence supported by this passage? Is this capture the same event as that story? How important is this story? A general chat model can answer them in prose, but prose must be parsed, costs output tokens, and invites the model to hedge.

A **typed judgement** asks a closed question with a schema (`choice` among labels with written criteria, or a `score`) and gets back a **distribution**: probability per label. Three properties make this a different tool from a desk:

- **cheap and fast**: input-only pricing, no long generation;
- **checkable**: the answer has a fixed shape, so it can be validated, aggregated, calibrated and replayed;
- **independent**: a judge from a different model family than the writer reduces correlated errors (a writer grading itself shares its blind spots).

Choir calls this "System One" after the fast, intuitive mode in Kahneman's dual-process model; the desks are System Two.

6.2 How Choir does it

The transport (code: `internal/gateway/jev.go`):

- The model is **pinned**: `typesafe/jev-1.13` on OpenRouter's Decisions alpha endpoint (`jevModel`, `openRouterDecisionsURL`, `:19–24`). The wire request has no model field; a caller cannot pick another one (`JudgmentRequest`, comment at `:26`).
- `validateJudgmentRequest` (`:153`) requires a JSON object `state` and named question objects of type `noul`, `choice` or `score`; anything else is refused before it costs money.
- `HandleJudgment` (`:258`) authenticates, enforces the Jev peer pin, picks a rate bucket by `scope` (`judgments` or `judgments:intake_labels`, so a busy poll cannot starve the support judge), and calls `Decide`.

- `Decide` (`:76`) appends a **receipt** (request and response digests) to a JSONL file before returning, and returns the provider's body verbatim: the distribution is "neither selected nor normalized here". Interpretation is the caller's job.
- The route is off unless `GATEWAY_JEV_JUDGMENTS_ENABLED` is true (`config.go`, default false). *Inference:* it is enabled on Node B through the provider env file, since the support judge runs there; the nix unit does not set it.

The guest client is `gatewayruntime.JudgmentClient` (`internal/gatewayruntime/jev.go`): the computer's bearer, no provider credential, the response kept as a raw distribution.

Three users:

1. **Citation support** (code: `internal/agentcore/support_judge.go`). After a Texture revision commits, `JudgeRevisionSupport` hands its new citation pairs to a background job and returns at once: the commit never waits for Jev. The question has two labels with written criteria ("yes: the cited passage states, or directly entails, what the sentence relies on it for"; `supportJudgeQuestion`, `:82–90`). Jev reads the committed sentence and the captured block, re-read and hash-checked, never text a desk supplied. Verdicts are recorded as a system act (`system:support`); a failure sends one record to the Texture, which wakes it to correct the citation. Concurrency is 2, queue wait 2 minutes, call timeout 90 s (`:57–61`). The job holds no durable obligation: a crash leaves pairs pending and nothing resumes at boot, in line with the restart rule (doc: support-judgement-to-jev design).
2. **Intake labels** for Autopaper: is this new capture the same event as a shortlisted story? Concurrency 4 (code: `internal/agentcore/intake_labels.go:44`, task text at `:281`).
3. **Front-page importance**, a host timer, not a guest (code: `internal/frontimportance/scorer.go`).

The design change on 14 October is a lesson in **moving work off a serial path**. A management turn spent 38 minutes hand-checking one Texture's citations while no story moved (doc: management-serial-turn-blocks-paper). Judging a listed pair needs no desk reasoning, so it became a bounded background job with its own budget, and management's turn went back to dispositions. The design names its falsifier: if Jev's verdicts are worse than management's, the failure-to-correction ratio will show it.

6.3 Where it is weak

- **One vendor, one alpha endpoint.** If OpenRouter's Decisions endpoint or the TypeSafe host changes, three features degrade together; there is no fallback judge.
- **Not every call is metered.** Support judgements append a `jev_judgment` cost event to the committing run; intake labels have no run to attach to and are unrecorded (named residual in the design), and front-importance calls bypass the gateway entirely (section 2.3).
- **A distribution needs a policy.** Returning the raw distribution is the right transport choice; it puts the threshold decision (what probability counts as "no") in each caller, where it must be written down and measured.

6.4 When to use a typed judge, and when not

use Jev when	use a desk when
the question is closed and its criteria fit in two sentences	the answer is an action or a piece of writing
the inputs are already selected (a sentence and a passage)	the work is to find the inputs
many instances arrive and each is independent	instances depend on each other's outcomes
an independent second opinion is the point	the judgement needs the desk's own context

7. The search plane

7.1 Primer: fan-out, tiers, caching and backoff

A search request can go to many providers. Four strategies:

- **fan-out**: ask everyone at once, merge; best recall and latency, worst cost;
- **sequential fallback**: ask one, then the next on failure; cheapest, slowest when the first is slow;
- **tiered escalation**: ask the cheap tier at once, add an expensive provider only when the cheap answers fall short; a middle path;
- **hedging**: ask a second provider only if the first is slow.

Three supporting ideas recur:

- **Request coalescing** ("singleflight"): if an identical request is already in flight, join it instead of starting another. The shared work must run on a context detached from the first caller, or the first caller leaving cancels everyone.
- **Per-class backoff**: a 429 rate limit clears in seconds, an exhausted quota in hours or days, a revoked key never. One backoff for all of them either hammers a dead account or benches a healthy one for a day.
- **Check-then-act races**: "if count < cap then count++ and call" is wrong under concurrency unless check and increment are one atomic step.

7.2 How Choir does it

Providers. Nine are configured on Node B: searxng (self-hosted), keenable, brave, serper, parallel, exa, tavily, serpapi and chatgpt web search (doc: the cooling record; code: `internal/gateway/search.go`, `search_chatgpt.go`). Each search provider call is bounded at 30 s.

The stream. `HandleSearch` (code: `handlers.go:775`) authenticates, takes a token from the computer's search bucket, starts the router, and writes NDJSON: a start line naming the providers started, reserved and cooling; one line per provider as it answers; a final line with "M of N answered", pending providers and escalations. Then it logs one line per search with `served=miss|hit|joined`, result counts, distinct URLs and how many the computer had already seen in the past day.

Cache and join (code: `internal/gateway/searchplane/cache.go`). The key is scope (the computer by default), the query with whitespace collapsed, and `max_results` (`cacheKey`, `:125`). An identical search within the TTL (30 minutes, 10 for queries with time words) is replayed from memory with no provider call; an identical search in flight is joined. The shared search runs on `context.WithoutCancel(ctx)` bounded by its own timeout (`:158`). Only arrivals with results are cached, so a failure never hides a later success. Memory is bounded at 64 MiB.

Health and cooldowns (code: `searchplane/policy.go`). `ClassifyCall` (`:117`) turns an error into one of: success, empty, rate-limited, quota-limited, auth error, server error, timeout, error. `DefaultBackoffPolicy` (`:33`) gives each class a base and a ceiling:

class	base cooldown	ceiling
rate limited	60 s	15 min
server error, timeout	2 min	30 min
other error	5 min	1 h
quota limited	1 h	6 h
empty results (after 10 in a row)	5 min	6 h
auth error	7 days	7 days

Backoff doubles per strike. Health is durable in SQLite (`search-health.db`), so a gateway restart does not forget which providers are out. A caller that went away is `caller_gone`, never a strike (`Router.call`, `router.go:634-660`), and empty results bench only after ten in a row (`EmptyStreakBeforeCooldown`), because one empty answer is usually the query's fault.

Budget routing (code: `searchplane/router.go`, `config.go:105-121`; doc: search report §4.1). Providers have tiers: keenable, searxng and brave are `free`; serper (cost 1), parallel (5), exa (6), tavily (8) and serpapi (15) are `paid`; chatgpt is `constrained` because it shares the ChatGPT usage limit with inference, so it goes last. An unknown provider is `paid` at cost 1000: it never spends by default. A search starts every eligible free provider; it escalates by one paid provider, cheapest first, when fewer than 10 distinct URLs have arrived and either every started call has answered or 8 seconds passed; at most 2 paid providers per search (4 for a `deep` search), none later than 20 s in. Daily caps (150 calls for serper, parallel, exa, tavily; 100 serpapi; 50 chatgpt) are counted per UTC day in `provider_daily_calls`. `ReserveCall` reads and increments under the store's mutex, so concurrent searches cannot overshoot (`sqlite_health.go:80`). A capped provider is skipped with reason `capped`, not counted as a failure. Rollback is two environment variables: `CHOIR_SEARCH_ROUTING=fanout` and `CHOIR_SEARCH_PROVIDER_DAILY_CAPS=none`.

7.3 Where it is weak

The measurements are in the search report and the cooling record (doc):

- **Volume and waste.** About 4,000 searches a day, two thirds from the Autopaper computer; exact repeats only 3.5%; 31% re-ask within ten minutes something the same computer just asked; about 1.5% of returned results are read and none are kept; 27% of page reads fail at the publisher.
- **Most of the pool was out.** On 14 October seven of nine providers were cooling: paid accounts out of credit (402, 432, "run out of searches"), brave flapping on a free plan's 1-request-per-second limit, and SearXNG returning nothing because its upstream engines block the datacenter address. Keenable carried the load (7,502 attempts, 7,417 successes). Fan-out at full health was estimated at about $155 a day; budget routing at about $1-3 a day expected and $4.50 worst case at the caps.
- **Classification by text.** `ClassifyCall` decides "quota" versus "rate limit" by phrases such as "run out" and "monthly" inside a 429 body. The same fragility as section 4.4, with money attached: a misclassified quota error retries every minute against an empty account.
- **A lazy-initialization race.** `SearchClient.ensurePlane` (`search_plane.go:117`) checks `c.plane != nil` without a lock, so the first concurrent searches after a start can build the plane twice (named residual in the search report). `sync.Once` is the standard fix.
- **Caps hold only for one gateway process.** The mutex is in-process. *Inference:* two gateway instances sharing the SQLite file would each hold their own mutex and could overshoot; a single SQL statement (`UPDATE ... SET calls = calls + 1 WHERE calls < cap`) would make the database the authority instead of the process.
- **No per-run budget.** The gateway sees the computer, not the run, so the "71-search run" the report targets cannot be bounded yet; the guest would have to send `run_id`.

7.4 Options (the report's sequence)

slice	what	expected effect	status at this commit
1	result cache and in-flight join	2.6-3.8% fewer searches; the instrument for duplication	landed (7250d36bf)
4	budget routing, tiers, caps	from about $155/day to about $1-3/day at full health	landed (0e8248b5c)
2	keep every search as an object (`choir.search_result`) with provenance edges	builds the object graph from searching; 50-100 MB/day	design
3	full-text index over captures and hits; read-through for recent URLs	about 15% fewer searches, 17% fewer fetches	design

slice	what	expected effect	status at this commit
horizon	local embeddings over the corpus	retrieval without the web for known material	sizing only

The order is a lesson in itself: the report shipped the smallest, contained change first because it is also the **instrument** that measures the problem, then the change with the largest money effect.

8. What it costs: metering, pricing and prompt caching

8.1 Primer

Metering counts usage; **billing** turns usage into charges; they need different trust. A meter for your own dashboard can live anywhere. A meter you bill from must live where the metered party cannot edit it.

List price is not marginal cost. Choir pays for inference three ways: per token at list rates, a flat subscription (ChatGPT; the OpenCode Go plan with per-model allowances), and free previews. A report "at list rates" answers "what would this cost at scale"; it does not answer "what did we pay".

Prompt caching. Providers cache the processed prefix of a prompt. A later request that starts with the same bytes (system prompt, then tools, then messages, in order) pays a much lower rate for the cached part: deepseek-v4.1-flash lists 0.30 per million input tokens and 0.006 cached; gpt-6-luna 0.10 and 0.01 (code: `pricingTable`, `internal/llmcost/pricing.go`; doc: model-prices). One changed byte early in the prompt invalidates everything after it. The design rule follows: **put what never changes first, and what changes last**.

8.2 How Choir does it

Token counts start at the provider. `provider.Usage` carries `InputTokens`, `OutputTokens`, `CachedInputTokens` (always a subset of input) and `CacheReported`, which distinguishes "zero cached" from "the provider did not say" (code: `internal/provider/provider.go:158–182`). The adapters read four different wire shapes for cached tokens (doc: prompt cache analysis, table). The gateway logs each call's tokens (`handlers.go`, "inference succeeded ... tokens=%d+%d cached=%d") and returns `Usage` to the guest.

In the guest, each tool-loop iteration emits a progress event with the model and tokens; `llmcost.ExtractCostEntries` reads events with phase `tool_loop`, `jev_judgment` or `execution` (code: `internal/llmcost/aggregator.go:58–95`), prices them with `EstimateCallWithCache` (`estimator.go:51`), and `internal/apihandler/costs.go` serves `/api/costs` with breakdowns by model, provider, run, trajectory and day, including cache hit rate and savings. A model with no known cached rate prices cached tokens at the input rate, never at zero (`cachedRate`, `pricing.go`), so the estimate errs high. The price table now covers the models Choir runs; the cost-track investigation's finding that it "only knows gpt-4 and claude-3" was true at `dde6900f` and has since been fixed.

Search is metered separately: per-provider daily call counts in the search health database, visible on the loopback health route.

8.3 Where it is weak

- **The meter lives in the metered party.** `/api/costs` is computed inside the guest from the guest's own run events. Fine for an owner's dashboard; not a basis for billing strangers. The gateway sees every call and keeps no durable ledger. *Inference:* the first pricing work will need a gateway-side ledger keyed by computer.
- **Gaps in coverage.** Intake-label Jev calls and the front-importance timer's calls are not in `/api/costs`; search spend is not in it at all.

- **List rates on flat routes.** In the last priced paper window the paper cost $0.33-0.65 in usage against $2.91-3.23 at list rates, 81% of the list cost on a model that was free for a limited time (doc: state-and-vision §9.2). Tokens run about 28:1 input to output. When free tiers end, cost at the same volume could rise severalfold.
- **Caching is defeated across turns.** Every desk's system prompt includes "Current UTC date/time" to the second near token 275, rebuilt on every activation (code: `internal/agentcore/tool_profiles.go:207`, template `internal/runtimeprompts/overlays/temporal_context.yaml:8`; still present at this commit). Everything after it misses: the rest of a 2,300- to 4,900-token system prompt, the tools, and the replayed history. Within one activation the loop is append-only and caches well; the ceiling there is about 70-84% of input tokens. The analysis lists ten more breakers (B2-B11), including per-run session affinity (`x-opencode-session` set to the run id, `provider.go:1121`) and no `prompt_cache_key` on Responses requests.

8.4 Options and trade-offs

option	gain	cost
move the timestamp and per-run text out of the system prompt into the first user message	every activation reuses the static prefix; resumed runs reuse their history while warm	none to the model: the time is still in context
cache affinity per desk (`prompt_cache_key`, session header = thread key)	higher hit rates on providers that route by key	depends on provider behaviour; measure first
sticky model per persistent desk	one cache per desk	see section 3.4
gateway-side usage ledger (append-only, per computer, per call)	an authoritative meter for pricing; covers every route	storage and retention on the host
route-aware pricing (subscription, allowance, free, list)	reports what was actually paid as well as list	needs plan data the repo does not hold

The analysis's own first recommendation is the right habit: **deploy the measurement and read it before restructuring anything.**

9. Putting it together

9.1 One inference call, end to end

```
 1. run start      modelpolicy.EnrichMetadata: role -> pool:default
                   ThreadKey -> FNV-1a -> member + ordered fallbacks
 2. tool loop      CallToolLoopProviderWithRetries (429 text: 5/20/60 s)
 3. guest client   gatewayruntime.Provider.call: up to 3 attempts;
                   gateway 429 + Retry-After waited (<= 10 x 1 min);
                   "circuit open" returns at once
 4. hostlink       refused dial resent for up to 3 min (deploy gap)
 5. gateway        authenticateAutoputer (hash + guest peer)
                   rate limiter (60 / min / computer)
                   resolveFromMultiProvider (catalog check)
 6. breakers       provider breaker (unreachable only)
                     model breaker (5 counted failures -> open 30 s)
 7. adapter        Responses / Chat Completions / Anthropic wire shape
                   sanitized errors; 4xx reason logged, redacted
 8. back up        Usage (input, output, cached, reported) -> guest
 9. on failure     IsModelFailoverError -> next pool member, once each
10. accounting     loop.progress event -> llmcost -> /api/costs
```

9.2 A debugging checklist for "a desk's model calls fail"

Following CLAUDE.md's rule to read the trace before attributing behaviour:

1. **Read the run's events first.** `model_failover` and `provider_rate_limit` retry events name the provider, model and error text. "The model chose not to" is a hypothesis until the ledger says otherwise.
2. **Find the error class.** `circuit open (model unhealthy)` is a model breaker; `(upstream unhealthy)` is the provider breaker (unreachable); `rate limit exceeded for autoputer` is *our* per-computer limiter, not the provider.

3. **Read the gateway journal for that computer.** `rejected request status=... reason=...` gives a 4xx cause; `inference succeeded ... cached=` gives tokens.
4. **Check breaker state** on the loopback route `/provider/v1/breakers` (operator header required).
5. **Ask whether one account or one provider is shared** by the failing roles: the policy file and the pool.
6. **Only then** decide whether it is a substrate problem (breaker keying, error contract, identity) or a symptom.

Glossary

term	meaning
Ambient authority	Being trusted for where you are (loopback, a subnet) rather than for what you hold.
Bearer token	A credential that works for whoever presents it.
Binding (credential)	Tying a credential to its holder (an address, a key, a certificate), so a stolen copy fails elsewhere.
Budget routing	Choir's search routing: free providers first, one paid provider at a time on shortfall, with daily caps.
Bulkhead	Isolating failure domains so one failing part cannot exhaust shared capacity.
Circuit breaker	A closed/open/half-open state machine that stops calls to a failing dependency for a while.
Cooldown (benching)	A search provider taken out of rotation for a time set by its failure class and strike count.
Credential disk	`credential.img`, an 8 MiB root-only ext4 image that carries a computer's gateway token and event envelope into the guest.
Deadline propagation	Passing the caller's remaining time down each hop so inner layers stop when the caller will.
Failover	Moving work to the next alternative after the primary fails.
Fate-sharing	Components that fail together because they depend on one thing.
FNV-1a	A fast non-cryptographic hash used to assign a thread to a pool member.
Hedging	Sending a second request after a delay and taking the first answer.
Jev	A pinned typed-judgement model (`typesafe/jev-1.13`) reached through OpenRouter's Decisions endpoint.
Last known good	Falling back to the most recent valid configuration when a new one fails to parse.
List price	A provider's published per-token rate, as opposed to what a subscription or free route actually charges.
NDJSON	Newline-delimited JSON: one JSON object per line, suited to streaming.
Pool	An ordered set of interchangeable (provider, model) pairs; a thread is hashed to one and fails over to the rest.
Prompt caching	Provider-side reuse of a processed prompt prefix, billed at a lower rate.
Retry amplification	Retries at several layers multiplying one logical call into many upstream calls.
`Retry-After`	An HTTP header telling a client how long to wait before retrying.
Singleflight	Coalescing identical in-flight requests into one shared execution.
Thread key	The stable key hashed to pick a pool member: the Texture agent id, else the trajectory id.
Typed judgement	A closed question with a schema whose answer is a probability distribution over labels or a score.

Self-check

1. A research run fails with `provider opencode-go: circuit open (upstream unhealthy)`. Which breaker is open, what kind of error opens it, and why would a cat-jam-free 429 burst *not* produce this message at this commit?
2. Why does the gateway's `providerFailureCounts` exclude a 400 but include a 408 and a 429? Give the incident that taught this.
3. A bearer token for computer A is copied to computer B. Which gateway routes accept it from B, which refuse it, and what single change would make all of them refuse it?
4. The guest search client has no per-attempt timeout. Why is that safe now, when a 30-second guest timeout was a bug on 11 October?

5. The paper's `/api/costs` shows a low cache hit rate on gpt-6-luna. Name two causes from the code and the one change you would make first.

Answer sketches

1. The provider-wide breaker, which counts only transport failures to reach the provider (`providerUnreachable`). A 429 counts against the *model* breaker for cat-jam-free and produces "provider opencode-go model cat-jam-free: circuit open (model unhealthy)"; deepseek's breaker stays closed. This message means opencode-go itself was unreachable.
2. A 400 means the provider rejected *our* request; it says nothing about the provider's health, and counting it let one computer's bad requests open the breaker for every computer (10 October, inference-breaker-trips-on-client-errors). 408 and 429 are the provider saying it is slow or overloaded.
3. Inference, search and the OpenAI-compatible route accept it (any Choir guest address passes). Jev refuses it, because the bearer is pinned to A's guest address. Generalizing the Jev pin to every route, with vmctl moving the pin at boot, makes all of them refuse it.
4. Search is asynchronous: the cell gets a stub at once and each provider's results arrive as frames. Nothing waits on the stream, so there is no client deadline to collide with the gateway's; the stream ends at the gateway's own bound or when the desk session ends.
5. The per-second timestamp in every desk's system prompt (B1) breaks every activation's prefix; luna's short idle TTL (B11), no `prompt_cache_key` (B7) and per-trajectory model hashing (B5) add to it. First change: move the timestamp out of the system prompt, after reading the measured hit rate by model.

Read these three files

1. **`internal/gateway/handlers.go`**: the chokepoint. Read `HandleInference` for the order of checks (authenticate, rate limit, decode, resolve, call, sanitize), `authenticateAutoputer` and the peer checks below it for what "identity" means here, and `HandleSearch` for a streaming handler that logs measurement, not just success.
2. **`internal/gateway/circuit_breaker.go`**: about 330 lines that encode three incidents. Read the type comment, then `execute`, `providerFailureCounts` and `providerUnreachable`; ask of every branch "whose fault is this error?".
3. **`internal/modelpolicy/model_policy.go`**: how a role becomes a model. Read `Load` (last known good), `EnrichMetadata`, `ThreadKey` and `Assign`, and the management-task pin; then open `internal/modelcatalog/catalog.go` beside it.

Runners-up: `internal/gateway/searchplane/router.go` and `policy.go` (budget routing and per-class backoff), `internal/gateway/config.go` (the identity registry), and `internal/toolregistry/toolloop.go` around `IsModelFailoverError`.

Sources and confidence

- **Read in code at 980ffaf4e:** everything cited with a path. Line numbers were checked at that commit and will drift.
- **From Choir documents (doc):** incident timelines and counts, search volumes and provider health, cost windows, latency figures and cache breaker analysis: choir-state-and-vision §3.6-3.7, §8.2, §9.2; search-efficiency-and-reuse; support-judgement-to-jev design; prompt-cache-analysis; pricing-and-economics-notes; cost-track investigation; computer isolation security review §3.5, §4.6; model-prices; and the problem records linked in sections 2 to 8.
- **Inferred, not tested:** that an expired bearer on a long-lived or snapshot-resumed computer is never re-issued (2.3); that the documentation-range peer check is low risk (2.3); that string error matching is the likeliest next breaker incident (4.4); that the lack of jitter causes synchronized retries (4.4); that daily caps would overshoot with two gateway processes (7.3); that Jev is enabled on Node B through the env file (6.2); and that billing will need a gateway-side ledger (8.3).

- **Drift noticed while writing:** the state-and-vision report says every desk runs on deepseek and luna; the default pool in code also contains cat-jam-free. The cost-track report's "price table only knows gpt-4 and claude-3" is no longer true. The isolation review flags vmctl's acceptance of `192.0.2.*`; the gateway's runtime peer check accepts all three documentation ranges as well.
- **Not covered:** the Bedrock, Z.AI and Fireworks adapters in detail; the OpenAI-compatible adapter's translation rules; the guest-side research tools (`internal/researchtools`) beyond search; user authentication (`internal/auth`), which is the browser's identity, not the guest's.